

UNIVERSIDAD PERUANA UNIÓN

Facultad de Ingeniería y Arquitectura

EP Ingeniería de Sistemas

LENGUAJE DE PROGRAMACIÓN II

ACTIVIDAD AUTÓNOMA

SESIÓN 8

Pruebas de dependencias entre
Categorías y Productos

Presentado por

Mishelle Osorio

Docente: Reyna Barreto Benjamin David

Ciclo IV · Semestre 2026-2

8 de octubre de 2026

Informe de resultados y hallazgos

Actividad autónoma · Sesión 8 · PharmaSoft · Ejecución en SPA localhost:4200 y API localhost:8080

B1. Resumen de ejecución

Se ejecutaron 17 de los 18 casos: 11 pasaron y 6 fallaron. B-04 no pudo ejecutarse porque el flujo de baja eliminó físicamente el producto, por lo que no fue posible preparar una categoría con productos inactivos. Los fallos describen defectos observados en la aplicación; no son errores de ejecución del estudiante.

Grupo	Ejecutados	Pasan	Fallan	No ejecutados
Altas (A-01 a A-08)	8	7	1	0
Cambios (C-01 a C-05)	5	2	3	0
Bajas (B-01 a B-05)	4	2	2	1
Total	17	11	6	1

La matriz y el documento de evidencias registran los resultados por ID. Las capturas de referencia están en EVIDENCIAS_ordenadas_vertical.docx.

B2. Fichas de hallazgos

H-01 · A-04 · Alta La API acepta altas en categorías inactivas

Resultado y evidencia: La SPA no mostró Vitaminas; Postman aceptó el alta y devolvió 201 Created. Evidencia: A-04-SPA y A-04-Postman.

Causa probable: ProductoServiceImpl.create llama a buscarCategoriaPorId, que solo comprueba existencia y no el estado de la categoría.

Corrección propuesta: En el servicio, exigir categoría activa antes de guardar y responder 409. Mantener el filtro de la SPA como ayuda de uso.

H-02 · C-02 · Alta La API permite mover productos a categorías inactivas

Resultado y evidencia: La SPA ocultó Vitaminas, pero PUT de producto 21 devolvió 200 OK y guardó esa relación. Evidencia: C-02.

Causa probable: ProductoServiceImpl.update también usa buscarCategoriaPorId sin validar estado activo.

Corrección propuesta: Validar en el servicio que el destino esté activo cuando se cambia la categoría; devolver 409 y conservar la categoría original si falla.

H-03 · C-03 · Media Se desactiva una categoría con productos activos

Resultado y evidencia: La actualización de Antibióticos ID 2 fue aceptada; Postman devolvió PUT 200 OK. Evidencia: C-03.

Causa probable: CategoriaServiceImpl.update asigna el estado recibido y guarda la entidad sin consultar productos activos asociados.

Corrección propuesta: Antes de desactivar, consultar si hay productos activos asociados y rechazar con 409. La regla debe estar en backend.

B2. Fichas de hallazgos (continuación)

H-04 · C-04 · Alta La API no revalida la categoría al guardar
Resultado y evidencia: Se desactivó la categoría con el formulario aún abierto; el POST fue aceptado con 201 Created. Evidencia: C-04.
Causa probable: La operación de alta vuelve a buscar la categoría, pero la búsqueda no valida su estado actual.
Corrección propuesta: Revalidar el estado dentro de la transacción de alta y rechazar el registro si la categoría está inactiva.
H-05 · B-01 · Alta La baja de producto es física y pierde el registro
Resultado y evidencia: DELETE /productos/27 devolvió 204; A07 desapareció y la consulta posterior dio 404. Se esperaba conservarlo inactivo. Evidencia: B-01.
Causa probable: ProductoServiceImpl.delete llama a productoRepository.delete(producto), que elimina la fila.
Corrección propuesta: Cambiar estado a false y guardar la entidad. Verificar que el listado conserve el registro y que las consultas de negocio respeten estado.
H-06 · B-02 · Media La segunda baja devuelve 404 en vez de 409
Resultado y evidencia: La segunda petición DELETE /productos/27 devolvió 404 Not Found; se esperaba 409 para un producto ya dado de baja. Evidencia: B-02.
Causa probable: La baja física de B-01 quitó el registro; el servicio solo puede tratarlo como recurso inexistente.
Corrección propuesta: Implementar primero la baja lógica; al recibir un producto existente con estado false, responder 409 con mensaje claro.

B3. Cobertura de reglas de dependencia

Regla	Dónde se valida hoy	Resultado observado
Existencia de categoría	API: ProductoServiceImpl.buscarCategoriaPorId; busca por ID y lanza 404 si no existe.	A-03 pasa.
Categoría activa al crear o cambiar producto	SPA: producto-form.ts filtra opciones. API: create/update no comprueban estado.	A-04 y C-02 fallan; C-04 confirma falta de revalidación.
Nombre único sin distinguir mayúsculas	API: ProductoServiceImpl.create usa existsByNombreIgnoreCase.	A-05 pasa; la unicidad en update no se cubrió.
No desactivar categoría con productos activos	CategoriaServiceImpl.update no consulta productos antes de guardar estado.	C-03 falla.
No borrar categoría con productos asociados	CategoriaServiceImpl.delete consulta existsByCategoriald.	B-03 pasa; B-04 no se ejecutó.
Baja lógica de producto	ProductoServiceImpl.delete usa productoRepository.delete.	B-01 y B-02 fallan: se elimina físicamente.

Los casos B-01, B-02 y B-04 describen consecuencias relacionadas del mismo comportamiento de baja física; se mantienen separados porque verifican resultados distintos.

B4. Recomendación para el filtro por categoría

El listado actual obtiene los productos con GET /api/v1/productos y filtra en el cliente. ProductoController.findAll devuelve un Iterable; ProductoServiceImpl.readAll usa findAll. Para catálogos grandes, conviene que el servidor aplique el filtro antes de paginar.

Endpoint: agregar un parámetro opcional categoriaId a GET /api/v1/productos. Cuando se envía, devolver solo los productos de esa categoría; cuando se omite, conservar el listado general.

GET /api/v1/productos?categoriaId=26&page=0&size=20

Repositorio y servicio: incorporar una consulta paginada derivada de la relación de producto, por ejemplo findByCategoria_Id(categoriaId, pageable), y usarla desde ProductoServiceImpl. Mantener el orden y el tamaño de página en el servicio.

Frontend: ProductoService.listar debe añadir categoriaId y paginación mediante HttpParams. ProductoList debe actualizar esos parámetros al cambiar el selector. La ruta abierta desde Categorías puede continuar usando queryParams categoriaId; el servidor realizará el filtrado definitivo.

Ventaja: se reciben únicamente registros de la categoría seleccionada y la paginación representa el conjunto filtrado completo, no solo los productos cargados en el navegador.

B5. Preguntas de análisis

1. ¿Por qué no basta con ocultar categorías inactivas en la SPA?

La regla de interfaz solo guía a quien usa el formulario. Postman o una petición concurrente pueden enviar directamente el ID inactivo. A-04, C-02 y C-04 demostraron que la API aceptó esas operaciones. La API debe ser la autoridad y validar el estado antes de guardar. La SPA debe conservar el filtro para mejorar la experiencia. Referencias: producto-form.ts, opcionesCategoria; ProductoServiceImpl.create/update.

2. ¿Qué debería ocurrir al desactivar una categoría con productos activos?

La propuesta es rechazar la desactivación con 409 mientras existan productos activos asociados. Así se evita que productos vendibles queden bajo una categoría que no puede seleccionarse en altas o cambios. C-03 mostró que CategoriaServiceImpl.update actualmente guarda el estado sin esa comprobación.

3. ¿Debe poder eliminarse una categoría que conserve productos históricos?

No. Para conservar referencias e informes, una categoría referenciada por productos debe mantenerse aunque esos productos estén inactivos. CategoriaServiceImpl.delete ya consulta existsByCategoriaId. B-04 no se ejecutó porque la baja física impidió preparar el escenario; por eso no se atribuye un resultado observado a esa prueba.

B5. Preguntas de análisis (continuación)

4. ¿Qué revela C-04 sobre el momento de validación?

El estado de la categoría puede cambiar después de abrir el formulario. La API debe revalidarlo en cada escritura, dentro de la transacción y lo más cerca posible del guardado. C-04 devolvió 201 después de desactivar la categoría, así que la validación por sí sola en la carga de la SPA no es suficiente. Si el sistema permite modificaciones concurrentes, debe proteger también la consistencia en la base de datos.

5. Si se corrige A-04 en el backend, ¿qué cambia en la SPA y qué sigue igual?

La SPA debe seguir ocultando las categorías inactivas al crear y ofrecer la categoría original al editar sin cambiarla, comportamiento observado en C-05. También debe mostrar claramente el error 409 si la categoría se desactiva después de cargar el formulario. La regla de integridad pasa a estar garantizada por `ProductoServiceImpl`; la selección y los mensajes siguen siendo responsabilidad de la interfaz.

Referencias al código revisado

Backend

`ProductoServiceImpl.java`: `create` (líneas 31-44), `buscarCategoriaPorId` (46-50), `update` (53-64), `delete` (77-81), `readAll` (85-90).

`CategoriaServiceImpl.java`: `update` (48-69), `delete` (74-86).

`ProductoController.java`: `findAll` (24-28).

`ProductoRepository.java`: `existsByNombreIgnoreCase` y `existsByCategoriald`.

Frontend

`producto-form.ts`: `opcionesCategoria` y validación al guardar (líneas 37-40, 67-72).

`producto-list.ts`: lectura de `categoriald` y filtro local (líneas 17, 32-39).

`producto-service.ts`: método `listar`.

Evidencia complementaria: `matriz-pruebas-s8.docx` y `EVIDENCIAS_ordenadas_vertical.docx`. Las respuestas HTTP y capturas de cada caso se identifican por su ID.

Nota de alcance: las fichas describen causas probables sustentadas en el código revisado y proponen correcciones; no afirman que esos defectos hayan sido corregidos en esta entrega.